CSA3713

SOFTWARE TESTING

CO1 – AT2 – Short Answers Test

S. Dharmika
192421094
B.Tech IT

1. Testing as an Engineering Activity

Testing should follow a systematic process with clear objectives, test cases, expected results, and proper documentation. Structured test design covers different scenarios, reduces missed defects, and improves software quality.

2. Importance of a Defined Testing Process

A defined testing process includes test planning, test case design, execution, defect reporting, and retesting. In a banking system, testing deposits, withdrawals, and balance updates systematically helps prevent transaction failures.

3. Testing as a Structured Process in SDLC

Testing should be performed throughout the SDLC, including requirement analysis, design, unit testing, integration testing, system testing, and acceptance testing. Following these stages helps identify defects early and ensures better software quality.

4. Error, Defect, and Failure

An error is a human mistake, a defect is a fault in the software code or design, and a failure occurs when the software behaves incorrectly during execution. For example, a developer enters an incorrect password-checking condition, causing the login system to accept invalid credentials.

5. Early Defect Detection

The principle of early testing states that testing should begin as early as possible in the SDLC. Finding an incorrect requirement during analysis is cheaper and easier than fixing the same problem after deployment.

6. Tester Involvement During Requirement Analysis

Testers can review healthcare requirements, identify ambiguities, and prepare test scenarios before coding begins. Early involvement prevents misunderstandings, reduces rework, and improves the accuracy and safety of the system.

7. Origin of Defects in the Software Lifecycle

Defects may originate from unclear requirements, design mistakes, coding errors, or incorrect integration. Clear and testable requirements help developers understand expected behaviour and prevent defects in payroll calculations.

8. Boundary Value Analysis

A crash caused by boundary inputs is an input-validation or boundary-handling defect, possibly missed due to inadequate testing. Boundary Value Analysis tests values at, just below, and just above limits to detect such problems.

9. Defect Repository

A defect repository stores information about previously identified defects, their causes, and their solutions. Testers can use this history to create test cases for recurring problems and improve defect detection in future releases.

10. Functional Defect in an ATM

An ATM withdrawal failure caused by incorrect transaction logic is a functional defect because the system does not perform its required operation correctly. Testing valid and invalid withdrawals, balance limits, and insufficient funds can help prevent it.

11. Defect Clustering

Defect clustering means that most defects are concentrated in a small number of modules. Testers should prioritize these modules because they are more likely to contain additional defects, improving testing efficiency.

12. Performance Defect

Slow performance under heavy user load is a non-functional performance defect. Load and stress testing help identify bottlenecks and enable developers to improve response time and system stability.

13. Historical Defect Data in Regression Testing

Historical defect data helps testers identify previously faulty modules and create regression test cases for them. Reusing these cases ensures that new changes do not introduce old defects and saves testing time.

14. Role of Testers in Collaboration

Testers should communicate regularly with developers, clarify requirements, and report defects with clear reproduction steps. Early collaboration helps identify integration problems sooner and reduces rework.

15. Defect Prevention Techniques

Defect prevention involves root cause analysis, requirement reviews, coding standards, and developer training. These practices eliminate the causes of recurring defects and improve software quality across releases.

16. Standardizing Mobile Application Testing

The team should follow a standard testing plan with documented test cases, defined procedures, and consistent defect reporting. Testing across devices, operating systems, and network conditions helps improve application reliability.

17. Testing vs Debugging

Testing identifies defects by executing or examining the software, whereas debugging locates the cause and fixes the defect. For example, testing identifies a mobile app crash, and debugging helps the developer find and correct the faulty code.

18. Cross-Browser Testing

Cross-browser testing checks whether a web application works correctly in browsers such as Chrome, Firefox, and Edge. Testers should verify layouts, forms, navigation, and functionality across browsers and fix compatibility issues.

19. Proper Validation in a Logistics Application

Testers should verify input formats, required fields, valid ranges, and consistency between modules. Applying positive, negative, and boundary test cases helps detect validation gaps and improves system reliability.

20. Systematic Test Design

Testing should be based on requirements, risk analysis, and structured test cases instead of random execution alone. Systematic test design provides better coverage, repeatable results, and more effective defect detection.

21. Data Consistency During Integration Testing

Testers should verify that data passed between healthcare modules remains accurate and consistent. Integration test cases should check data flow, interfaces, database updates, and error handling to identify the root cause of inconsistencies.

22. Tester Involvement Across SDLC Phases

Testers review requirements during analysis, evaluate designs, prepare test cases during development, and perform testing during integration and deployment. Early involvement identifies defects sooner, improves collaboration, and reduces development costs.

23. Test Coverage During Peak Usage

The team should identify missing scenarios and perform load, stress, and concurrency testing. Better test coverage verifies system behaviour under heavy traffic and helps prevent crashes during peak usage.

24. Defect Repository and Documentation

A defect repository should record the defect ID, description, severity, steps to reproduce, status, and resolution. Proper documentation makes tracking easier, supports root cause analysis, and prevents defects from being repeated.

25. Improving Banking System Reliability

The team should follow a structured process involving requirement reviews, unit testing, integration testing, system testing, and regression testing. Testing deposits, withdrawals, balances, and transaction failures improves reliability and prevents financial errors.

26. Edge Cases and Boundary Testing

Testers should include minimum, maximum, just-below-limit, just-above-limit, and invalid inputs instead of testing only normal values. Boundary testing helps identify crashes, incorrect calculations, and validation defects near input limits.

27. Process-Based Testing and Standardization

A standard testing process should define common test plans, test case formats, execution procedures, and defect reporting methods for every module. This ensures consistent testing practices, better coverage, and uniform software quality.

28. Testing Unexpected Inputs

Testers should check empty fields, invalid formats, special characters, excessively long inputs, and out-of-range values. Proper input validation and negative testing prevent crashes and make the application more robust.

29. Reducing Defect Leakage

Defect leakage occurs when defects escape testing and reach production. Requirement reviews, risk-based testing, regression testing, and root cause analysis help identify missed defects and reduce future production failures.

30. Early Defect Detection and Efficiency

The team should introduce testing during requirement analysis and continue it throughout the SDLC. Early defect detection reduces correction costs, avoids rework, saves time, and improves overall software quality.